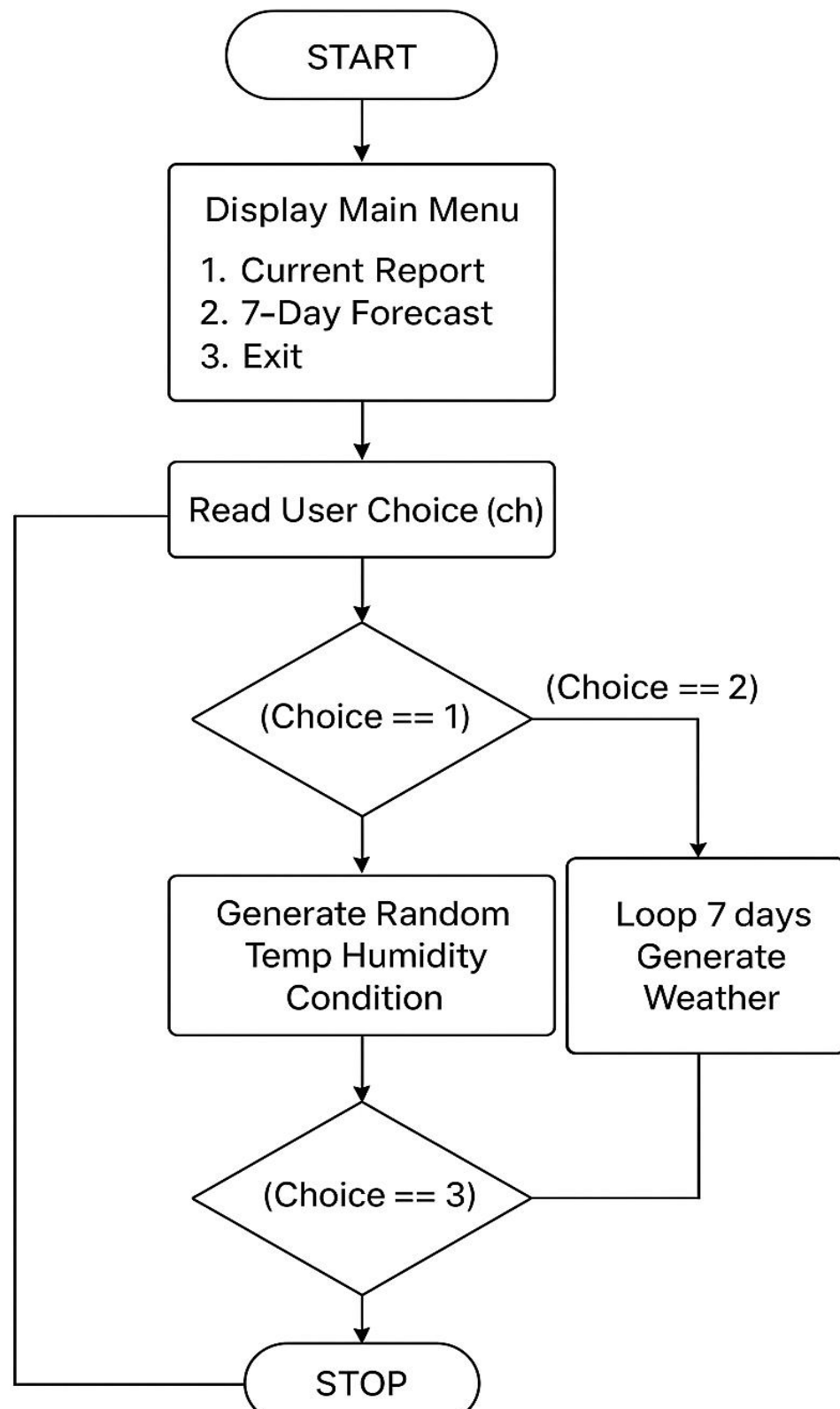


- Conditional statements to link Temperature to Weather Conditions (e.g., Low Temp + Rain = Snow).
2. Arithmetic calculations are used to convert units (Celsius to Fahrenheit) and calculate averages.
 3. String manipulation is used to map numeric condition codes to readable text (e.g., 1 = "Sunny").
 4. Intermediate steps, such as storing data in Structures (struct Weather), are handled internally to ensure organized output.
 5. After processing, the formatted weather report is displayed clearly on the screen for the user.
 6. The menu reappears automatically, enabling the user to perform multiple checks without restarting the program.
12. A loop structure keeps the program active until the user chooses the "Exit" option from the menu; upon exit, the program terminates safely.

FLOWCHART

WEATHER REPORT SIMULATOR



PROGRAM

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
struct Weather {
```

```
    int temperature;
```

```
    int humidity;
```

```
    char condition[20];
```

```
};
```

```
// Function to generate weather condition text
```

```
void getCondition(int temp, int humidity, char cond[]) {
```

```
    if (temp < 0 && humidity > 50)
```

```
        sprintf(cond, "Snow");
```

```
    else {
```

```
        int c = rand() % 4;
```

```
        switch(c) {
```

```
            case 0: sprintf(cond, "Sunny"); break;
```

```
            case 1: sprintf(cond, "Cloudy"); break;
```

```
            case 2: sprintf(cond, "Rainy"); break;
```

```
case 3: sprintf(cond, "Stormy"); break;
```

```
}
```

```
}
```

```
}
```

```
void currentReport() {
```

```
    struct Weather w;
```

```
    w.temperature = (rand() % 41) - 5; // -5 to 35
```

```
    w.humidity = rand() % 101;        // 0 to 100
```

```
    getCondition(w.temperature, w.humidity, w.condition);
```

```
    printf("\n----- CURRENT WEATHER REPORT -----\\n");
```

```
    printf("Temperature : %d deg C\\n", w.temperature);
```

```
    printf("Humidity   : %d%%\\n", w.humidity);
```

```
    printf("Condition   : %s\\n", w.condition);
```

```
    printf("-----\\n");
```

```
}
```

```
void weeklyForecast() {
```

```
    struct Weather week[7];
```

```
    int i;
```

```
    printf("\n----- 7-DAY WEATHER FORECAST -----\\n");
```

```

for ( i = 0; i < 7; i++) {
    week[i].temperature = (rand() % 41) - 5;
    week[i].humidity = rand() % 101;
    getCondition(week[i].temperature, week[i].humidity, week[i].condition);

    printf("Day %d: Temp = %d deg C, Humidity = %d%%, Condition = %s\n",
        i + 1,
        week[i].temperature,
        week[i].humidity,
        week[i].condition);
}

printf("-----\n");
}

int main() {
    int choice;
    srand(time(NULL));

    while (1) {
        printf("\n===== WEATHER SIMULATOR =====\n");
        printf("1. Current Weather Report\n");
    }
}

```

```
printf("2. 7-Day Forecast\n");

printf("3. Exit\n");

printf("Enter your choice: ");

scanf("%d", &choice);


switch(choice) {

    case 1: currentReport(); break;

    case 2: weeklyForecast(); break;

    case 3:

        printf("\nThank you for using Weather Simulator!\n");

        exit(0);

    default:

        printf("Invalid choice! Please try again.\n");

}

}

return 0;

}
```

OUTPUT

```
===== WEATHER SIMULATOR =====
1. Current Weather Report
2. 7-Day Forecast
3. Exit
Enter your choice: 1
```

```
----- CURRENT WEATHER REPORT -----
Temperature : 16 deg C
Humidity    : 59%
Condition   : Cloudy
-----
```

```
===== WEATHER SIMULATOR =====
1. Current Weather Report
2. 7-Day Forecast
3. Exit
Enter your choice: 2
```

```
----- 7-DAY WEATHER FORECAST -----
Day 1: Temp = 34 deg C, Humidity = 19%, Condition = Cloudy
Day 2: Temp = 29 deg C, Humidity = 79%, Condition = Cloudy
Day 3: Temp = 0 deg C, Humidity = 11%, Condition = Cloudy
Day 4: Temp = 19 deg C, Humidity = 40%, Condition = Sunny
Day 5: Temp = 16 deg C, Humidity = 39%, Condition = Rainy
Day 6: Temp = 8 deg C, Humidity = 95%, Condition = Cloudy
Day 7: Temp = 11 deg C, Humidity = 28%, Condition = Rainy
-----
```

```
===== WEATHER SIMULATOR =====
1. Current Weather Report
2. 7-Day Forecast
3. Exit
Enter your choice: 3
```

```
Thank you for using Weather Simulator!
```

CONCLUSION

In conclusion, this project successfully demonstrates the practical application of C programming concepts in implementing a Weather Report Simulator using data structures. It strengthens the understanding of how environmental data can be modeled and how computers internally process stochastic data.

The modular, menu-driven design of the program enhances clarity, usability, and maintainability, making it suitable for learning as well as practical use. Through the development of this project, key concepts such as algorithm design, modular functions, and logical problem-solving were deeply reinforced. Overall, the project provides a solid foundation for further studies in simulation modelling , computer architecture, and advanced programming techniques.

FUTURE ENHANCEMENTS

1. Implement support for real-time data fetching via APIs (e.g., Open Weather Map) , enhancing the practical use of the program.
2. Add a graphical user interface (GUI) using libraries like GTK or a web-based interface for improved user experience.
3. Integrate error-handling systems to detect invalid inputs for custom temperature ranges and guide the user with meaningful messages.
4. Include file handling features to save generated reports, generate logs, and maintain history for academic or project documentation.

REFERENCES

Book reference:

- “DATA STRUCTURES USING C”, A K SHARMA, PEARSON EDUCATION 2013
- “THE C PROGRAMMING LANGUAGE”, KERNIGHAN & RITCHIE, PRENTICE HALL

Web references:

- <https://www.w3schools.com>
- <https://www.tutorialspoint.com>
- <https://www.javatpoint.com>
- <https://www.geeksforgeeks.org>